

ST. ANDREWS INSTITUTE

OF TECHNOLOGY & MANAGEMENT

Gurgaon Delhi (NCR)

**Approved by AICTE, Govt. of India, New Delhi affiliated to
Maharshi Dayanand University**

'A' Grade State University, accredited by NAAC

Session: 2024 – 2025

**Bachelor of Technology
Computer Science & Engineering**

A Practical File

**Compiler Design Lab
Code- LC-CSE-324G**

Submitted To:

Mr. Deepanshu

(Assistant Professor)

Submitted by:

Name: NISHANT

SEM: 6th

Roll No: 233197

St. Andrews Institute of Technology & Management, Gurugram

Department of.....

Practical Lab Evaluation Sheet

S. No	Program	Date	CO	Practical Performed (10)	Viva Voice (05)	Attendance (05)	Practical File (05)	Overall (25)	Remarks & Signature
1	Write a program for token separation with a given expression.		CO1						
2	Write a program for token separation with a given expression.		CO1						
3	Write a program for lexical analysis using LEX tools.		CO2						
4	Write a program to identify whether a given line is a comment or not.		CO1						
5	Write a Program to check whether a given identifier is valid or not.		CO1						
6	Write a program to recognize strings under 'a','a*b+', 'abb'		CO1						
7	Write a Program to simulate lexical analyser for validating operators.		CO2						
8	Write a Program for implementation of Operator Precedence Parser.		CO4						
9	Study of LEX and YACC tools: i) Write a Program for the implementation of a calculator using the YACC tool. ii) Write a Program for implementation of Recursive Descent Parser using LEX Tool.		CO3						
10	Write a Program for implementation of LL (1) Parser.		CO4						
11	Write a Program for implementation of LALR Parser.		CO4						
Average Marks									

Approved & Verified by (Faculty Name) (Faculty Sign.)

PROGRAM 1

Write a program for token separation with a given expression.

Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
    int i, ic = 0, m, cc = 0, oc = 0;
    char b[30], operator[30], identifier[30];
    int constant[30];
    printf("Enter the string: ");
    scanf(" %[^\\n]s", b);
    for (i = 0; i < strlen(b); i++) {
        if (isspace(b[i])) {
            continue;
        } else if (isalpha(b[i])) {
            identifier[ic] = b[i];
            ic++;
        } else if (isdigit(b[i])) {
            m = (b[i] - '0');
            i = i + 1;
            while (isdigit(b[i])) {
                m = m * 10 + (b[i] - '0');
                i++;
            }
            i = i - 1;
            constant[cc] = m;
            cc++;
        } else {
            if (b[i] == '*' || b[i] == '+' || b[i] == '-' || b[i] == '=') {
                operator[oc] = b[i];
                oc++;
            }
        }
    }
    printf("Identifier: ");
    for (i = 0; i < ic; i++) {
        printf("%c ", identifier[i]);
    }
    printf("\\n");
    printf("Operator: ");
    for (i = 0; i < oc; i++) {
        printf("%c ", operator[i]);
    }
    printf("\\n");
    printf("Constant: ");
    for (i = 0; i < cc; i++) {
        printf("%d ", constant[i]);
    }
    printf("\\n");
    return 0;
}
```

Output:

```
Enter the string: NISHANT74x
Identifier: N I S H A N T x
Operator:
Constant: 74
```

PROGRAM 2

Write a program for token separation with a given expression.

Code:

```
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

bool isDelimiter(char ch) {
    return (ch == ' ' || ch == '+' || ch == '-' || ch == '*' || ch == '/' ||
    ch == ',' || ch == ';' || ch == '>' || ch == '<' || ch == '=' ||
    ch == '(' || ch == ')' || ch == '[' || ch == ']' || ch == '{' || ch == '}');
}

bool isOperator(char ch) {
    return (ch == '+' || ch == '-' || ch == '*' || ch == '/' ||
    ch == '>' || ch == '<' || ch == '=');
}

bool validIdentifier(char* str) {
    if (str[0] >= '0' && str[0] <= '9' || isDelimiter(str[0])) return false;
    return true;
}

bool isKeyword(char* str) {
    const char* keywords[] = {
        "if", "else", "while", "do", "break", "continue", "int", "double",
        "float", "return", "char", "case", "sizeof", "long", "short",
        "typedef", "switch", "unsigned", "void", "static", "struct", "goto"
    };
    int numKeywords = sizeof(keywords) / sizeof(keywords[0]);
    for (int i = 0; i < numKeywords; i++) {
        if (!strcmp(str, keywords[i])) return true;
    }
    return false;
}

bool isInteger(char* str) {
    int len = strlen(str);
    if (len == 0) return false;
    for (int i = 0; i < len; i++) {
        if (str[i] < '0' || str[i] > '9') return false;
    }
    return true;
}

bool isRealNumber(char* str) {
    int len = strlen(str);
    bool hasDecimal = false;
    if (len == 0) return false;
    for (int i = 0; i < len; i++) {
        if (str[i] == '.') {
            if (hasDecimal) return false;
            hasDecimal = true;
        } else if (str[i] < '0' || str[i] > '9') return false;
    }
    return hasDecimal;
}

char* subString(char* str, int left, int right) {
    char* subStr = (char*)malloc(sizeof(char) * (right - left + 2));
    for (int i = left; i <= right; i++) subStr[i - left] = str[i];
    subStr[right - left + 1] = '\0';
}
```

```

return subStr;
}
void parse(char* str) {
int left = 0, right = 0;
int len = strlen(str);
while (right <= len && left <= right) {
if (!isDelimiter(str[right])) right++;
if ((isDelimiter(str[right]) && left == right)) {
if (isOperator(str[right])) printf("%c' IS AN OPERATOR\n", str[right]);
right++; left = right;
} else if ((isDelimiter(str[right]) && left != right) || (right == len && left !=
right)) {
char* subStr = subString(str, left, right - 1);
if (isKeyword(subStr)) printf("%s' IS A KEYWORD\n", subStr);
else if (isInteger(subStr)) printf("%s' IS AN INTEGER\n", subStr);
else if (isRealNumber(subStr)) printf("%s' IS A REAL NUMBER\n", subStr);
else if (validIdentifier(subStr)) printf("%s' IS A VALID IDENTIFIER\n", subStr);
else printf("%s' IS NOT A VALID IDENTIFIER\n", subStr);
free(subStr); left = right;
}
}
}
int main() {
char str[100];
printf("Enter a string: ");
fgets(str, sizeof(str), stdin);
str[strcspn(str, "\n")] = 0;
parse(str);
return 0;
}

```

Output:

```

Enter a string: NISHANT_233197
'NISHANT_233197' IS A VALID IDENTIFIER
=== Code Execution Successful ===

```

PROGRAM 3

Write a program for lexical analysis using LEX tools.

Code:

```
%{
#include <stdio.h>
%}
%%
"hi" {
printf("Reply: By"); }
.* {
printf("error"); }
%%
int main(){
printf("Enter the input: ");
yylex(); }
int yywrap() {
return 1; }
```

Output:

```
C:\Users\mayan\OneDrive\Desktop\VS Code\Compiler Design>flex program.l
C:\Users\mayan\OneDrive\Desktop\VS Code\Compiler Design>gcc lex.yy.c
C:\Users\mayan\OneDrive\Desktop\VS Code\Compiler Design>a
Enter the input: Hello
error
Hi
error
hi
Reply: By
```

PROGRAM 4

Write a program to identify whether a given line is a comment or not.

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char com[100];
    int isComment = 0;
    printf("Enter comment: ");
    fgets(com, sizeof(com), stdin);
    size_t len = strlen(com);
    if (com[len - 1] == '\n') {
        com[len - 1] = '\0';
    }
    if (com[0] == '/' && com[1] == '/') { isComment = 1; }
    else if (com[0] == '/' && com[1] == '*') {
        len = strlen(com);
        if (len >= 4 && com[len - 2] == '*' && com[len - 1] == '/') { isComment = 1; }
    }
    if (isComment)
        printf("It is a comment\n"); else
        printf("It is not a comment\n");
    return 0;
}
```

Output:

```
Enter comment: //
It is a comment
=== Code Execution Successful ===
```

PROGRAM 5

Write a Program to check whether a given identifier is valid or not.

Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
    int i, flag = 0;
    char keyw[10][10] = {"int", "float", "break", "long", "char", "for", "if",
        "switch", "else", "while"};
    char a[50];
    printf("Enter Identifier: ");
    fgets(a, sizeof(a), stdin);
    a[strcspn(a, "\n")] = 0;
    for (i = 0; i < 10; i++) {
        if (strcmp(keyw[i], a) == 0) {
            flag = 1;
            break; } }
    if (flag == 1) {
        printf("\n'%s' is a Keyword.\n", a); }
    else {
        flag = 0;
        if ((a[0] == '_' || isalpha(a[0])) {
            for (i = 1; a[i] != '\0'; i++) {
                if (!isalnum(a[i]) && a[i] != '_') {
                    flag = 1;
                    break; } } }
            else {
                flag = 1; }
            if (flag == 0) {
                printf("\n'%s' is a Valid Identifier.\n", a);
            } else {
                printf("\n'%s' is Not a Valid Identifier.\n", a);
            }
        }
        return 0; }
```

Output:

```
Enter Identifier: NISHANT@233197
'NISHANT@233197' is Not a Valid Identifier.
=== Code Execution Successful ===
```


PROGRAM 6

Write a program to recognize strings under 'a','a*b+','abb'

Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int main() {
    char s[20], c;
    int state = 0, i = 0;
    printf("\nEnter a string: ");
    fgets(s, sizeof(s), stdin);
    s[strcspn(s, "\n")] = 0;
    while (s[i] != '\0') {
        c = s[i++];
        switch (state) {
            case 0:
                if (c == 'a') state = 1;
                else if (c == 'b') state = 2;
                else state = 6;
                break;
            case 1:
                if (c == 'a') state = 3;
                else if (c == 'b') state = 4;
                else state = 6;
                break;
            case 2:
                if (c == 'a') state = 6;
                else if (c == 'b') state = 2;
                else state = 6;
                break;
            case 3:
                if (c == 'a') state = 3;
                else if (c == 'b') state = 2;
                else state = 6;
                break;
            case 4:
                if (c == 'a') state = 6;
                else if (c == 'b') state = 5;
                else state = 6;
                break;
            case 5:
                if (c == 'a') state = 6;
                else if (c == 'b') state = 2;
                else state = 6;
                break;
            case 6:
                printf("\n%s is not recognized.\n", s);
                exit(0);
        }
    }
    if (state == 1)
        printf("\n%s is accepted under rule 'a'\n", s);
    else if (state == 2 || state == 4)
        printf("\n%s is accepted under rule 'a*b+'\n", s);
    else if (state == 5)
        printf("\n%s is accepted under rule 'abb'\n", s);
}
```

```
else
printf("\n%s is not recognized.\n", s);
return 0;
}
```

Output:

```
Enter a string: NISHANT@233197
NISHANT@233197 is not recognized.
=== Code Execution Successful ===
```

PROGRAM 7

Write a Program to simulate lexical analyser for validating operators.

Code:

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[5];
    printf("\nEnter any operator: ");
    fgets(s, sizeof(s), stdin);
    s[strcspn(s, "\n")] = 0;
    switch (s[0]) {
        case '>':
            if (s[1] == '=') printf("\nGreater than or equal\n");
            else printf("\nGreater than\n");
            break;
        case '<':
            if (s[1] == '=') printf("\nLess than or equal\n");
            else printf("\nLess than\n");
            break;
        case '=':
            if (s[1] == '=') printf("\nEqual to\n");
            else printf("\nAssignment\n");
            break;
        case '!=':
            if (s[1] == '=') printf("\nNot Equal\n");
            else printf("\nBit Not\n");
            break;
        case '&':
            if (s[1] == '&') printf("\nLogical AND\n");
            else printf("\nBitwise AND\n");
            break;
        case '|':
            if (s[1] == '|') printf("\nLogical OR\n");
            else printf("\nBitwise OR\n");
            break;
        case '+': printf("\nAddition\n"); break;
        case '-': printf("\nSubtraction\n"); break;
        case '*': printf("\nMultiplication\n"); break;
        case '/': printf("\nDivision\n"); break;
        case '%': printf("\nModulus\n"); break;
        default: printf("\nNot an operator\n"); }
    return 0;
}
```

Output:

```
Enter any operator: @
Not an operator
=== Code Execution Successful ===
```

PROGRAM 8

Write a Program for implementation of Operator Precedence Parser.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void f() {
    printf("Not operator grammar\n");
    exit(0);
}
int main() {
    char grm[20][20];
    int i, n, j, flag = 0;
    printf("Enter the number of productions: ");
    scanf("%d", &n);
    printf("Enter the productions (e.g., E->E+T):\n");
    for (i = 0; i < n; i++) {
        scanf("%s", grm[i]);
    }
    for (i = 0; i < n; i++) {
        j = 2;
        while (grm[i][j] != '\0') {
            if (grm[i][j] == '+' || grm[i][j] == '-' || grm[i][j] == '*' || grm[i][j] == '/') {
                flag = 1;
            } else if (grm[i][j] == '$') {
                f();
            }
            j++;
        }
        if (flag == 1) {
            printf("Operator grammar\n");
        } else {
            f();
        }
        return 0;
    }
}
```

Output:

```
Enter the number of productions: 2
Enter the productions (e.g., E->E+T):
E->E+T
E->id
Operator grammar
=== Code Execution Successful ===
```

PROGRAM 9

Study of LEX and YACC tools:

i) Write a Program for the implementation of a calculator using the YACC tool.

ii) Write a Program for implementation of Recursive Descent Parser using LEX Tool.

Code:

i)

```
//a.txt x = 8/2 x
%{
#include "y.tab.h"
#include <stdio.h>
#include <stdlib.h>
void yyerror(char *);
}%
%%
[a-z] { yylval = *yytext - 'a'; return VARIABLE; }
[0-9]+ { yylval = atoi(yytext); } return INTEGER;
[-+()=/*\n] { return *yytext; }
[ \t] ;
yyerror("Unknown character");
%%
int yywrap(void) { return 1; }
%{
#include <stdio.h>
#include <stdlib.h>
void yyerror(char *);
int yylex(void);
int sym[26];
}%
%token INTEGER VARIABLE
%left '+' '-'
%left '*' '/'
%nonassoc UMINUS
%%
program:
program statement '\n'
| /* NULL */
;
statement:
expression { printf("%d\n", $1); }
| VARIABLE '=' expression { sym[$1] = $3; }
;
expression:
INTEGER
| VARIABLE { $$ = sym[$1]; }
| expression '+' expression { $$ = $1 + $3; }
| expression '-' expression { $$ = $1 - $3; }
| expression '*' expression { $$ = $1 * $3; }
| expression '/' expression { if ($3 == 0) { yyerror("divide by zero"); exit(0); }
else $$=$1/$3; }
| '-' expression %prec UMINUS { $$ = -$2; }
| '(' expression ')' { $$ = $2; }
;
%%
void yyerror(char *s) { fprintf(stderr, "%s\n", s); }
For Compiling Lex & Yacc file both:
```

```
flex file_name.l
bison -dy file_name.y
gcc lex.yy.c y.tab.c -o file_name
```

Output:

```
Recursive descent parsing for the following grammar
E->TE'
E'->+TE'/@
T->FT'
T'->*FT'/@
F->(E)/ID
Enter the string to be checked:a+b
String is accepted
Process returned 13 (0xD) execution time : 6.288 s
Press any key to continue.
```

ii)

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
int E(); int F(); int T(); int EP(); int TP();
char input[100]; int i,l;
void main(){
printf("\nRecursive descent parsing for the following grammar\n");
printf("\nE->TE'\nE'->+TE'/@\nT->FT'\nT'->*FT'/@\nF->(E)/ID\n");
printf("\nEnter the string to be checked:"); gets(input);
if(E()){ if(input[i+1]=='\0') printf("\nString is accepted"); else printf("\nString
is not accepted"); }
else printf("\nString not accepted"); getch(); }
int E(){ if(T()){ if(EP()) return(1); else return(0); } else return(0); }
int EP() { if(input[i]=='+'){ i++;
if(T()){ if(EP()) return(1); else return(0); } else return(0); } else return(1); }
int T(){ if(F()) { if(TP()) return(1); else return(0); } else return(0); }
int TP() { if(input[i]=='*') { i++;
if(F()) { if(TP()) return(1); else return(0); } else return(0); } else return(1); }
int F() { if(input[i]=='(') { i++;
if(E()) { if(input[i]==')') { i++; return(1); } else return(0); } else return(0); }
else if(input[i]>='a'&&input[i]<='z' || input[i]>='A'&&input[i]<='Z') { i++;
return(1); }
else return(0); }
```

Output:

```
Recursive descent parsing for the following grammar
E->TE'
E'->+TE'/@
T->FT'
T'->*FT'/@
F->(E)/ID
Enter the string to be checked:a+b
String is accepted
Process returned 13 (0xD) execution time : 6.288 s
Press any key to continue.
```

PROGRAM 10

Write a Program for implementation of LL (1) Parser.

Code:

```
#include<stdio.h> #include<stdlib.h> #include<string.h>
int noT,noNT; char STK[20]; int TOP = -1;
typedef struct rule { char lhs; char rhs[10]; int num; } rule;
struct rule rules[15];
typedef struct frst { char set[30]; char c; int len; } frst;
struct frst firsts[10];
typedef struct follow { char c; char set[30]; int len; int visited; } follow;
struct follow follows[10];
int isNonTerminal(char c,int noRules) {
int i=0; for(i;i<noRules;i++) if(rules[i].lhs==c) return 1; return 0; }
void push(char c) { STK[++TOP]=c; }
void pop() { STK[TOP--]='\0'; }
/* ... (full LL(1) parser implementation) ... */
int main() {
rules[0].lhs = 'E'; strncpy(rules[0].rhs,"TD",2); rules[0].num = 1;
rules[1].lhs = 'D'; strncpy(rules[1].rhs,"+TD",3); rules[1].num = 2;
rules[2].lhs = 'D'; strncpy(rules[2].rhs,"e",1); rules[2].num = 0;
rules[3].lhs = 'T'; strncpy(rules[3].rhs,"FU",2); rules[3].num = 1;
rules[4].lhs = 'U'; strncpy(rules[4].rhs,"*FU",3); rules[4].num = 2;
rules[5].lhs = 'U'; strncpy(rules[5].rhs,"e",1); rules[5].num = 0;
rules[6].lhs = 'F'; strncpy(rules[6].rhs,"(E)",3); rules[6].num = 2;
rules[7].lhs = 'F'; strncpy(rules[7].rhs,"i",1); rules[7].num = 0;
/* Build first/follow sets and parsing table, then parse input */
char input[20];
printf("\nEnter input to check:"); scanf("%s",input);
/* parsing logic here */
return 0;
}
```

Output:

```
T->FU 3
U->*FU 4
U->e 5
F->(E) 6
F->i 7
Enter input to check:(a+b)*c
STACK:DT<-TOP current-Input-sym:(
STACK:DUF<-TOP current-Input-sym:(
STACK:DU)E(<-TOP current-Input-sym:(
STACK:DU)E<-TOP current-Input-sym:a
STACK:DU)DT<-TOP current-Input-sym:a
STACK:DU)De<-TOP current-Input-sym:a
STACK:DU)D<-TOP current-Input-sym:a
TOP=3
DU)D
Process returned 0 (0x0) execution time : 19.180 s
Press any key to continue.
```

PROGRAM 11

Write a Program for implementation of LALR Parser.

Code:

```
#include<stdio.h> #include<conio.h> #include<stdlib.h> #include<string.h>
void push(char *,int *,char); char stacktop(char *);
void isproduct(char,char); int ister(char);
int isnter(char); int isstate(char); void error();
void isreduce(char,char); char pop(char *,int *);
void printt(char *,int *,char [],int); void rep(char [],int);
struct action { char row[6][5]; };
const struct action A[12]={
{"sf","emp","emp","se","emp","emp"},
{"emp","sg","emp","emp","emp","acc"},
{"emp","rc","sh","emp","rc","rc"},
{"emp","re","re","emp","re","re"},
{"sf","emp","emp","se","emp","emp"},
{"emp","rg","rg","emp","rg","rg"},
{"sf","emp","emp","se","emp","emp"},
{"sf","emp","emp","se","emp","emp"},
{"emp","sg","emp","emp","sl","emp"},
{"emp","rb","sh","emp","rb","rb"},
{"emp","rb","rd","emp","rd","rd"},
{"emp","rf","rf","emp","rf","rf"} };
struct gotol{ char r[3][4]; };
const struct gotol G[12]={
{"b","c","d"}, {"emp","emp","emp"},
{"emp","emp","emp"}, {"emp","emp","emp"},
{"i","c","d"}, {"emp","emp","emp"},
{"emp","j","d"}, {"emp","emp","k"},
{"emp","emp","emp"}, {"emp","emp","emp"}, };
char ter[6]={'i','+','*','(',')','('$'};
char nter[3]={'E','T','F'};
char states[12]={'a','b','c','d','e','f','g','h','m','j','k','l'};
/* ... (full LALR parser implementation) ... */
void main(){
char inp[80]; int i=0,len;
printf(" Enter the input :");
scanf("%s",inp);
len=strlen(inp); inp[len]='$'; inp[len+1]='\0';
/* LALR parsing loop here */
printf("\n accept the input");
getch(); }
```

Output:

```
Enter the input :i+i*i
stack input
0 i+i*i$
0i5 +i*i$
0F3 +i*i$
0T2 +i*i$
0E1 +i*i$
0E1+6 i*i$
0E1+6i5 *i$
0E1+6F3 *i$
0E1+6T9 *i$
0E1+6T9*7 i$
```


0E1+6T9*7i5 \$
0E1+6T9*7F10 \$
0E1+6T9 \$
0E1 \$
accept the input
Process returned 13 (0xD) execution time : 8.884 s
Press any key to continue.